

1 Abstract

This report details the evaluation of data integrity and signal quality for high-frequency time-series chromatographic data, specifically focusing on the 'UV_1.280_mAU' signal. The primary objective was to quantify Signal-to-Noise (SNR) ratios, assess baseline stability, and identify data artifacts such as sensor drift or negative volume outliers that could compromise product safety and process reliability. The analysis workflow involved rigorous preprocessing, including the exclusion of rows with missing 'Fraction_unique_ID's and the application of baseline correction to stabilize the signal. A critical step involved identifying outlier runs based on high intra-run noise variance (exceeding the mean plus two standard deviations) and subsequently screening these flagged runs for negative volume outliers. The results indicate that while the dataset contains runs with statistically significant noise or drift, no negative volume outliers were detected within the high-variance subset. This suggests that the primary data integrity issues in this subset are related to signal noise or sensor drift rather than physically impossible volume readings, reinforcing the need for continued noise monitoring and baseline correction.

2 Introduction

In the context of high-performance liquid chromatography (HPLC), the integrity of time-series data is paramount for ensuring accurate fraction collection and downstream quality control. This analysis focuses on the 'UV_1.280_mAU' variable, selected as the primary signal source due to its superior data completeness (94%), while explicitly excluding 'UV_2.260_mAU' due to significant missingness (70%). The dataset presents challenges, including the near-total absence of critical process parameters like 'Injection_ml' (99.9% missing), which necessitates reliance on internal standards or Total Ion Current (TIC) scaling rather than injection volume for normalization. Furthermore, the presence of negative values in 'volume_ml' and UV signals indicates potential data integrity issues that must be addressed prior to calculating SNR. The motivation for this specific analysis workflow is to evaluate the stability of the baseline and the presence of excessive noise or sensor drift across available runs. By quantifying the Signal-to-Noise (SNR) ratio and assessing baseline stability, this study aims to identify data artifacts that could compromise the accuracy of the chromatographic process, thereby supporting robust process reliability and product safety.

3 Methodology

The data analysis followed a structured three-step plan designed to ensure data quality before statistical evaluation. Step 1 focused on Data Integrity and Preprocessing. The raw data from 'chromatography_combined.csv' was loaded, and the 'Unnamed: 0' column was renamed to 'Fraction_number'. Rows were filtered to exclude any entries where 'Fraction_unique_ID' was missing to prevent skewing noise metrics. Additionally, negative 'volume_ml' values were flagged if they were less than -5 mL. Baseline correction was applied to the 'UV_1.280_mAU' variable to stabilize the signal, while 'UV_2.260_mAU' was excluded from general quality metrics. Step 2 involved Noise Variance Analysis and Drift Detection. Intra-run noise variance (standard deviation of baseline segments) was calculated for each 'run'. Temporal drift was assessed by plotting noise variance against 'Fraction_number' for the first five runs sorted by 'run_no'. The Signal-to-Noise (SNR) was calculated using the ratio of the median peak height to the baseline noise. Step 3 dedicated itself to Outlier Run Identification. Runs were identified as outliers if their intra-run noise variance exceeded the mean plus two standard deviations. Within these flagged runs, the count of 'volume_ml' values less than -5 mL was tallied to determine the prevalence of negative volume outliers. The final output included a summary of filtered row counts, statistical summaries of 'volume_ml' and 'UV_1.280_mAU' before and after correction, and a detailed table of flagged outlier runs with their respective variance and negative outlier counts.

4 Results and Discussion

The analysis successfully executed the data integrity plan, yielding specific insights into the quality of the chromatographic data. As illustrated in Figure 1, the variance-based filtering method effectively isolated

specific runs as outliers; these runs exhibited intra-run noise variances that significantly exceeded the threshold of the mean plus two standard deviations. This indicates the presence of runs with compromised signal quality, likely due to sensor drift or high-frequency noise. However, a critical finding emerged from the subsequent screening of these flagged runs: the count of 'volume_ml' values less than -5 mL was zero. This result indicates that while the dataset contains runs with statistically significant noise or drift, these anomalies do not manifest as physically impossible negative volume readings in the filtered dataset. The absence of negative volume outliers suggests that the primary data integrity issues in this specific subset are related to signal noise or sensor drift rather than volume measurement errors or data entry artifacts. Consequently, the reliance on 'volume_ml' for outlier detection was methodologically valid despite the high missingness in other process parameters, as volume data was available for the flagged runs. The results reinforce the necessity of maintaining baseline correction and noise monitoring protocols (Steps 2 and 3) even when immediate data entry errors like negative volumes are absent, as the variance metric itself serves as a robust indicator of compromised run quality.

5 Conclusion

In conclusion, this data analysis workflow provided a comprehensive evaluation of the integrity and quality of the high-frequency time-series chromatographic data. By prioritizing the 'UV_1.280_mAU' signal and applying rigorous preprocessing steps including baseline correction and missing data filtering, the study successfully quantified signal quality metrics. The key finding is that while the dataset exhibits runs with excessive noise and temporal drift, these issues do not correlate with negative volume outliers. This distinction is crucial for process reliability, as it allows for the identification of runs requiring further investigation regarding sensor stability without flagging them for data entry errors. The methodology proved effective in isolating high-variance runs, confirming that noise monitoring is a vital component of data quality assurance in this chromatographic context. Future work should continue to monitor these variance metrics to ensure that signal stability remains consistent across subsequent batches, thereby safeguarding the accuracy of fraction collection and downstream quality control decisions.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```

###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/data_integrity_report.md'

    # Load data
    df = pd.read_csv(input_path)

    # Rename 'Unnamed: 0' to 'Fraction_number'
    df.rename(columns={'Unnamed: 0': 'Fraction_number'}, inplace=True)

    # Check if 'run' column exists as per plan variables
    if 'run' not in df.columns:
        raise ValueError("Column 'run' is missing from the dataset.")

    # Filter rows where 'Fraction_unique_ID' is missing

```

```

df_clean = df.dropna(subset=['Fraction_unique_ID'])

# Check if 'UV_1_280_mAU' exists before proceeding with UV calculations
if 'UV_1_280_mAU' not in df_clean.columns:
    raise ValueError("Column 'UV_1_280_mAU' is missing from the dataset.")

# Flag negative 'volume_ml' values only if < -5 mL
df_clean['volume_flagged'] = df_clean['volume_ml'] < -5

# Check if 'volume_flagged' column exists (it was just created, so it should,
# but for safety)
if 'volume_flagged' not in df_clean.columns:
    raise ValueError("Column 'volume_flagged' was not created successfully.")

# Safe baseline calculation: check row count before slicing
n_rows = len(df_clean)
if n_rows == 0:
    raise ValueError("No data rows available after filtering.")

# Use min(n_rows, 1000) to ensure at least one row is taken if fewer than 1000
# exist
sample_size = min(n_rows, 1000)
baseline = df_clean['UV_1_280_mAU'].head(sample_size).median()

df_clean['UV_1_280_mAU_corrected'] = df_clean['UV_1_280_mAU'] - baseline

# Exclude 'UV_2_260_mAU' (drop column if present)
if 'UV_2_260_mAU' in df_clean.columns:
    df_clean = df_clean.drop(columns=['UV_2_260_mAU'])

# Calculate Summary Stats in a single pass
stats_before = df_clean[['volume_ml', 'UV_1_280_mAU']].agg(['min', 'max', 'mean'])
stats_before.columns = ['min', 'max', 'mean']

stats_after = df_clean[['volume_ml', 'UV_1_280_mAU_corrected']].agg(['min', 'max', 'mean'])
stats_after.columns = ['min', 'max', 'mean']

# Find Top 20 worst negative outlier indices
# Use boolean indexing directly to avoid intermediate DataFrame creation
# Ensure indices are valid before sorting
if len(df_clean.loc[df_clean['volume_flagged'], 'volume_ml']) > 0:
    top_20_outliers = df_clean.loc[df_clean['volume_flagged'], 'volume_ml'].sort_values(ascending=True).index[:20].tolist()
else:
    top_20_outliers = []

# Generate Markdown Report
report_lines = [
    "# Data Integrity & Preprocessing Report\n",
    f"## 1. Count of Filtered Rows\n",
    f"- Rows with missing 'Fraction_unique_ID' excluded: {len(df)} - {len(df_clean)}\n",
    f"- Final dataset size: {len(df_clean)} rows\n",
    "\n",
    "## 2. Summary Statistics\n",
    "\n",
    "**Before Baseline Correction**\n",

```

```

f"|_Variable_|_Min_|_Max_|_Mean_|\\n",
f"|-----|-----|-----|-----|\\n",
f"|_volume_ml_|_{stats_before.loc['volume_ml', '_min']:.4f}|_{stats_before.loc['volume_ml', '_max']:.4f}|_{stats_before.loc['volume_ml', '_mean']:.4f}|\\n",
f"|_UV_1_280_mAU_|_{stats_before.loc['UV_1_280_mAU', '_min']:.4f}|_{stats_before.loc['UV_1_280_mAU', '_max']:.4f}|_{stats_before.loc['UV_1_280_mAU', '_mean']:.4f}|\\n",
"\\n",
"**After_Baseline_Correction**\\n",
f"|_Variable_|_Min_|_Max_|_Mean_|\\n",
f"|-----|-----|-----|-----|\\n",
f"|_volume_ml_|_{stats_after.loc['volume_ml', '_min']:.4f}|_{stats_after.loc['volume_ml', '_max']:.4f}|_{stats_after.loc['volume_ml', '_mean']:.4f}|\\n",
f"|_UV_1_280_mAU_(Corrected)|_{stats_after.loc['UV_1_280_mAU_corrected', '_min']:.4f}|_{stats_after.loc['UV_1_280_mAU_corrected', '_max']:.4f}|_{stats_after.loc['UV_1_280_mAU_corrected', '_mean']:.4f}|\\n",
"\\n",
"##_3_Top_20_Worst_Negative_Outlier_Indices\\n",
f"|_Rank_|_Index_|_Volume_(mL)|\\n",
f"|-----|-----|-----|\\n"
]

for i, idx in enumerate(top_20_outliers, 1):
    vol_val = df_clean.loc[idx, 'volume_ml']
    report_lines.append(f"|_{i}|_{idx}|_{vol_val:.4f}|\\n")

report_content = "".join(report_lines)

# Save to file
with open(output_path, 'w') as f:
    f.write(report_content)

print(f"Report_saved_to_{output_path}")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

def Call_Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Guard clause: Check if required columns exist
    required_columns = ['run', 'UV_1_280_mAU', 'volume_ml']
    missing_columns = [col for col in required_columns if col not in df.columns]
    if missing_columns:
        raise ValueError(f"Missing_required_columns:_{missing_columns}")

    # Guard clause: Check if dataframe has at least one row
    if len(df) == 0:
        raise ValueError("Dataframe_is_empty.")

    # Select relevant columns

```

```

df_subset = df[['run', 'UV_1_280_mAU', 'volume_ml']]

# Check data distribution per run to handle single-row cases
rows_per_run = df_subset.groupby('run').size()
print("Rows per run:")
print(rows_per_run)

# Calculate variance per run, handling single-row runs by setting to 0
df_subset['variance'] = df_subset.groupby('run')['UV_1_280_mAU'].var()
df_subset['variance'] = df_subset['variance'].fillna(0)

# Guard clause: Check if 'variance' column is not empty or all NaN
if df_subset['variance'].isna().all() or df_subset['variance'].isna().sum() ==
    len(df_subset):
    raise ValueError("Variance column is empty or all NaN.")

# Calculate global mean and std for variance
global_mean_var = df_subset['variance'].mean()
global_std_var = df_subset['variance'].std()

# Identify runs with high intra-run noise (mean + 2*std)
threshold = global_mean_var + 2 * global_std_var
df_subset['is_flagged'] = df_subset['variance'] > threshold

# Filter flagged runs
flagged_runs = df_subset[df_subset['is_flagged']]

# Guard clause: Ensure flagged_runs is not empty before proceeding
if len(flagged_runs) == 0:
    # Create empty output if no flagged runs
    output_table = pd.DataFrame(columns=['run', 'variance', 'negative_outlier_count'])
    total_negative_outliers = 0
else:
    # Check if 'volume_ml' contains numeric values and no NaNs before applying
    lambda
    if not pd.api.types.is_numeric_dtype(flagged_runs['volume_ml']) or
        flagged_runs['volume_ml'].isna().any():
        raise ValueError("'volume_ml' column must contain numeric values without NaN in flagged runs.")

    # Count negative volume_ml outliers within flagged runs
    flagged_runs['negative_outlier_count'] = flagged_runs['volume_ml'].apply(
        lambda x: (x < -5).sum())

    # Prepare output table (top 10)
    output_table = flagged_runs[['run', 'variance', 'negative_outlier_count']]
    output_table.sort_values('negative_outlier_count', ascending=False).head(10).
    reset_index(drop=True)

    # Calculate total negative outliers across all flagged runs
    total_negative_outliers = flagged_runs['negative_outlier_count'].sum()

# Create output directory if it doesn't exist
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

# Generate Markdown content
md_content = "# Outlier Run Identification Results\n\n"

```

```

md_content += "##_Flagged_Runs_Table\n\n"
md_content += "|_run_|_variance_|_negative_outlier_count_|_\\n"
md_content += "|-----|-----|-----|\\n"

if len(output_table) > 0:
    for index, row in output_table.iterrows():
        md_content += f"|_{row['run']}|_{row['variance']:.4f}|_{int(row['negative_outlier_count'])}|_\\n"

md_content += f"\n##_Summary\n\n"
md_content += f"Total_count_of_negative_'volume_ml'_values_across_all_flagged_runs:_{int(total_negative_outliers)}**\\n"

# Save to file
output_file = os.path.join(output_dir, 'outlier_run_identification.md')
with open(output_file, 'w') as f:
    f.write(md_content)

print(f"Results_saved_to_{output_file}")

if __name__ == "__main__":
    Call_Code_Updates()

```

Listing 2: Code.3